

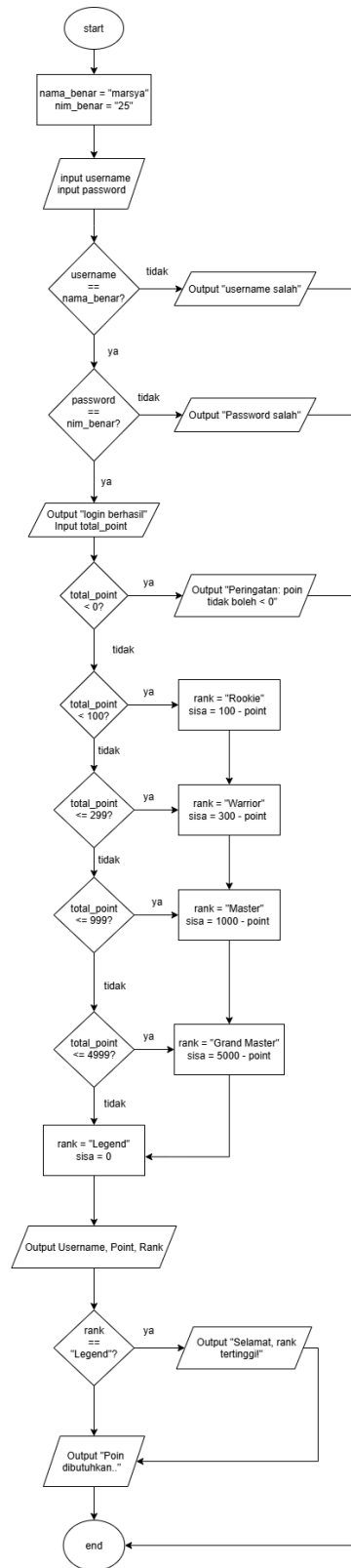
**LAPORAN PRAKTIKUM**  
**POSTTEST 2**  
**ALGORITMA PEMROGRAMAN DASAR**



**Disusun oleh:**  
**Cecilia Marsya Pua (2509106125)**  
**Kelas (B2 '26)**

**PROGRAM STUDI INFORMATIKA**  
**UNIVERSITAS MULAWARMAN**  
**SAMARINDA**  
**2026**

# 1. Flowchart



*gambar 1.1 Flowchart*

## 2. Deskripsi Singkat Program

Program ini dibuat untuk menentukan tingkatan (*rank*) seorang pemain game berdasarkan total poin yang dimiliki, sekaligus menghitung sisa poin yang dibutuhkan agar pemain bisa naik ke rank selanjutnya.

### Cara Kerja Program:

1. **Proses Login:** Sebelum masuk ke menu utama, pengguna diharuskan untuk login terlebih dahulu. Saya menerapkan percabangan bersarang (*nested if*) pada bagian ini. Pengecekan pertama dilakukan pada variabel *username* (harus diisi *marsya*), lalu jika benar akan dilanjutkan dengan pengecekan *password* (harus diisi 2 digit NIM 25). Jika ada data login yang tidak sesuai, program akan memberi tahu letak kesalahannya (*username salah* atau *password salah*).
2. **Input dan Validasi Poin:** Jika login berhasil, pengguna diminta untuk menginputkan total poin yang dimiliki. Sebelum menentukan *rank*, program akan mengecek nilainya terlebih dahulu sebagai bentuk *error handling*. Jika poin yang diisi kurang dari 0 (angka negatif), program akan menampilkan peringatan bahwa poin tidak boleh negatif.
3. **Penentuan Rank:** Jika poin bernilai 0 atau lebih, program akan mengecek tingkatan *rank* menggunakan struktur *if-elif-else*:
  - Poin di bawah 100 mendapat *rank* **Rookie**.
  - Poin 100 - 299 mendapat *rank* **Warrior**.
  - Poin 300 - 999 mendapat *rank* **Master**.
  - Poin 1000 - 4999 mendapat *rank* **Grand Master**.
  - Poin 5000 ke atas mendapat *rank* **Legend**.

**Output Hasil:** Program akan menampilkan *username*, total poin, dan rank saat ini. Khusus untuk pemain yang belum Legend, program akan menampilkan sisa poin yang kurang untuk naik rank. Namun jika pemain sudah mencapai rank Legend, program akan memberikan ucapan selamat karena sudah mencapai rank tertinggi.

### 3. Source Code

```
# login

nama_benar = "marsya"

nim_benar = "25"


print("=== LOGIN SYSTEM ===")

username = input("Masukkan username: ")

password = input("Masukkan Password: ")


# nested if

if username == nama_benar:

    if password == nim_benar:

        print("Login berhasil")

        print()


# input total poin

total_point = float(input("Masukkan total point: "))


# error handling jika poin kurang dari 0

if total_point < 0:

    print("Peringatan: total_point tidak boleh kurang dari 0!")

else:

    # penentuan rank dan sisa poin

    if total_point < 100:
```

```

        rank = "Rookie"

        sisa_point = 100 - total_point

    elif total_point <= 299:

        rank = "Warrior"

        sisa_point = 300 - total_point

    elif total_point <= 999:

        rank = "Master"

        sisa_point = 1000 - total_point

    elif total_point <= 4999:

        rank = "Grand Master"

        sisa_point = 5000 - total_point

    else:

        rank = "Legend"

        sisa_point = 0

print()

print("=== STATUS RANK PLAYER ===")

print("Username:", username)

print("Total Point:", total_point)

print("Rank Saat Ini:", rank)

# menampilkan penentuan sisa poin

if rank == "Legend":

    print("Selamat! Anda telah mencapai rank tertinggi!")

```

```

        else:

            print("Poin dibutuhkan untuk naik rank:", sisa_point)

    else:

        print("Password salah")
else:

    print("username salah")

```

#### 4. Hasil Output

```

=== LOGIN SYSTEM ===
Masukkan username: marsya
Masukkan Password: 25
login berhasil

Masukkan total point: 50

=== STATUS RANK PLAYER ===
Username: marsya
Total Point: 50.0
Rank Saat Ini: Rookie
Poin dibutuhkan untuk naik rank: 50.0

```

*gambar 4.1*

pengguna menginputkan total poin 50. Karena nilainya kurang dari 100, program menetapkan rank sebagai Rookie. Program juga menghitung sisa poin yang dibutuhkan untuk naik ke rank berikutnya ( $100 - 50 = 50.0$ ).

```

=== LOGIN SYSTEM ===
Masukkan username: marsya
Masukkan Password: 25
login berhasil

Masukkan total point: 150

=== STATUS RANK PLAYER ===
Username: marsya
Total Point: 150.0
Rank Saat Ini: Warrior
Poin dibutuhkan untuk naik rank: 150.0

```

*gambar 4.2*

Pengguna memasukkan total poin 150. Nilai ini memenuhi kondisi  $100 \leq poin \leq 299$ , sehingga pemain mendapatkan rank Warrior. Sisa poin untuk naik ke rank Master adalah  $300 - 150 = 150.0$ .

```

=== LOGIN SYSTEM ===
Masukkan username: marsya
Masukkan Password: 25
login berhasil

Masukkan total point: 450

=== STATUS RANK PLAYER ===
Username: marsya
Total Point: 450.0
Rank Saat Ini: Master
Poin dibutuhkan untuk naik rank: 550.0

```

*gambar 4.3*

Pengguna memasukkan total poin 450. Nilai ini masuk dalam rentang 300 sampai 999, sehingga rank yang diperoleh adalah Master. Sisa poin yang dibutuhkan untuk naik rank Grand Master adalah  $1000 - 450 = 550.0$ .

```

=== LOGIN SYSTEM ===
Masukkan username: marsya
Masukkan Password: 25
login berhasil

Masukkan total point: 2500

=== STATUS RANK PLAYER ===
Username: marsya
Total Point: 2500.0
Rank Saat Ini: Grand Master
Poin dibutuhkan untuk naik rank: 2500.0

```

*gambar 4.4*

Pengguna memasukkan total poin 2500. Karena poin berada di antara 1000 dan 4999, program menetapkan rank Grand Master. Poin yang kurang untuk mencapai rank Legend adalah  $5000 - 2500 = 2500.0$ .

```
=== LOGIN SYSTEM ===  
Masukkan username: marsya  
Masukkan Password: 25  
login berhasil  
  
Masukkan total point: 6000  
  
=== STATUS RANK PLAYER ===  
Username: marsya  
Total Point: 6000.0  
Rank Saat Ini: Legend  
Selamat! Anda telah mencapai rank tertinggi!
```

*gambar 4.5*

pengguna memasukkan total poin 6000. Karena poin bernilai 5000 atau lebih, program menetapkan rank Legend. Karena ini merupakan rank tertinggi, program menampilkan ucapan selamat tanpa menampilkan sisa poin.

```
=== LOGIN SYSTEM ===  
Masukkan username: marsya  
Masukkan Password: 25  
login berhasil  
  
Masukkan total point: -20  
Peringatan: total_point tidak boleh kurang dari 0!
```

*gambar 4.6*

Pengguna memasukkan nilai poin negatif (-20). Program mendeteksi bahwa nilai tersebut kurang dari 0, lalu menampilkan pesan peringatan "Peringatan: total\_point tidak boleh kurang dari 0!" dan menghentikan penentuan rank.

```
=== LOGIN SYSTEM ===  
Masukkan username: axel  
Masukkan Password: 25  
username salah
```

*gambar 4.7*

Pengguna memasukkan username yang salah (alex). Program langsung menghentikan proses pada kondisi if awal dan mencetak pesan "username salah".

```
=== LOGIN SYSTEM ===  
Masukkan username: marsya  
Masukkan Password: 52  
Password salah
```

*gambar 4.8*

Pengguna memasukkan username benar (marsya) tetapi password salah (52). Percabangan bersarang (nested if) memvalidasi bahwa username sesuai, lalu mengeksekusi blok else dari kondisi password untuk mencetak pesan "Password salah".

## 5. Git Add

```
PS C:\Users\MyASUS\OneDrive\文档\PRAKTIKUM APD 26> git add .
```

Perintah `git add .` digunakan untuk menambahkan semua perubahan file yang ada di dalam folder proyek ke dalam staging area Git. Staging area adalah tempat sementara di mana perubahan file disiapkan sebelum benar-benar disimpan ke dalam riwayat repository melalui perintah `git commit`.

## 6. Git Commit

```
PS C:\Users\MyASUS\OneDrive\文档\PRAKTIKUM APD 26> git commit -m "Menambahkan posttest3 Genap"  
[main 5d4cc77] Menambahkan posttest3 Genap  
2 files changed, 55 insertions(+), 1 deletion(-)  
create mode 100644 posttest/posttest-apd-3/tempCodeRunnerFile.py
```

Commit dalam Git dapat diibaratkan seperti menyimpan catatan atau rekaman atas perubahan yang telah dilakukan pada proyek. Setiap kali kita melakukan commit, Git akan menyimpan kondisi file yang sudah dimasukkan ke staging area sebagai satu titik riwayat baru. Titik riwayat ini nantinya bisa dilihat, dilacak, bahkan dikembalikan lagi jika dibutuhkan.

## 7. Git Push

```
PS C:\Users\MyASUS\OneDrive\文档\PRAKTIKUM APD 26> git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 501 bytes | 71.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/cecilpua/praktikum-apd-2026.git
 952279e..5d4cc77  main -> main
```

Git Push adalah perintah yang digunakan untuk mengirimkan atau mengunggah commit dari repository lokal ke repository remote, misalnya ke GitHub, GitLab, atau Bitbucket. Dengan melakukan push, semua perubahan yang sudah kita simpan melalui commit di komputer akan tersalin ke repository yang ada di server sehingga bisa diakses oleh orang lain atau digunakan pada perangkat lain.